Post-training (continued); BERT and Masked Language Models

Natural Language Processing, Week 8, Summer 2026

Dr. Paul Nulty

Lecturer

School of Computing and Mathematical Sciences

Birkbeck, University of London

Applying Large Language Models

- A pre-trained language model with no further training or tuning is sometimes referred to as a “base model”
- Depending on the contents of the pre-training corpus, there is no reason that a base model will answer questions or have a useful consistent behaviour or tone --- it simply continues the input tokens in the most likely way it would be continued in the training corpus
- To make models useful for conversation, question answering, or other tasks like document classification further training/tuning is necessary

Base model continuations

Prompt: Explain the moon landing to a six year old in a few sentences.
Output: Explain the theory of gravity to a 6 year old.

Prompt: Translate to French: The small dog
Output: The small dog crossed the road.

Applying Large Language Models

- This further training or tuning can be broken roughly into two types: **model alignment** and **instruction tuning**
- Alignment broadly refers to efforts to keep the responses of the LLM aligned with the goals of the user and more broad societal goals (e.g. deterring toxic language, not helping with weapons development)
- Instruction tuning has the goal of training LLMs to respond to objectives other than simply producing the most likely next word (e.g. conversation, question-answering, information retrieval, translation, text classification)

Types of fine-tuning

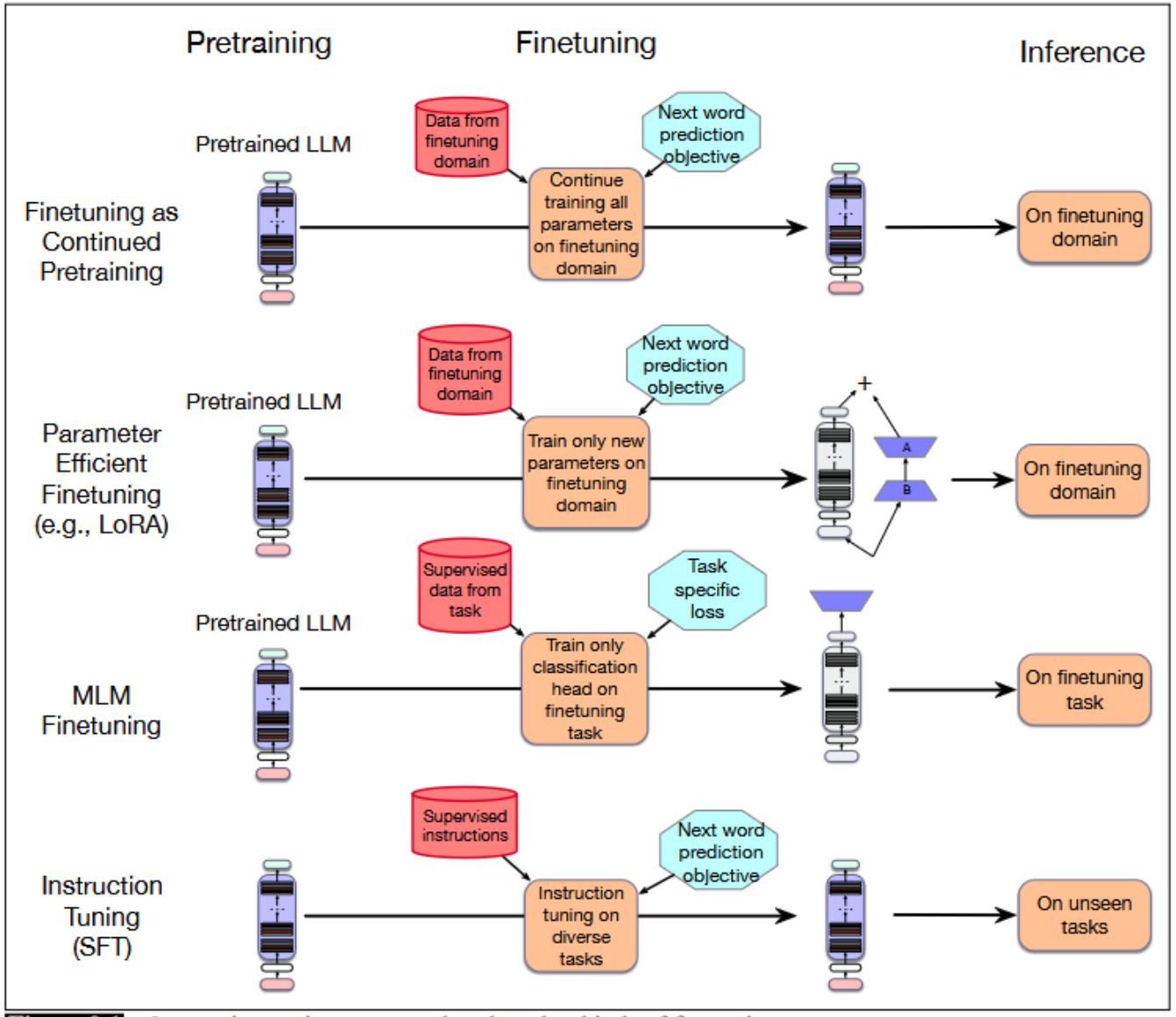


Figure 9.1 Instruction tuning compared to the other kinds of finetuning.

Information Retrieval and Question answering

- Many of the real-world use cases of LLM-based conversational agents are some form of question-answering or more broadly information retrieval
- The LLM may answer questions or retrieve data given a prompt for which the natural continuation is the answer to the question
- We can also further train the model parameters to give higher weights to outputs that respond in a conversation or answer a question
- And we can allow the model to retrieve text or embeddings at query-time to include local context or data that is continually updated: **Retrieval Augmented Generation**

Instruction tuning (SFT: supervised fine tuning)

Simple idea: we continue training the model on them using the same language modeling objective used to train the original model.

Essentially, finetuning the model on a corpus of instructions and responses

Instruction tuning

- May use parameter efficient fine tuning (e.g. LORA)
- Instructions followed by answers as Training Data: literally continuing training on material that looks like useful user queries and responses

Instruction tuning data

- Can be manually created
- Or curated question-answering data
- Or se examples of instructions and answers found online (in the wild)
- Use AI to paraphrase/augment this kind of data
- (another possibility: train on user-created data?); or data that naturally has instruction-response format.
- E.g.: https://github.com/tatsu-lab/stanford_alpaca#data-release

When should compound words be written as one word, with hyphens, or with spaces?

Ask Question

Asked 15 years ago Modified 3 years, 5 months ago Viewed 58k times

122

+300

In English, there are three types of compound words:

1. **the closed form**, in which the words are melded together, such as firefly, secondhand, softball, childlike, crosstown, redhead, keyboard, makeup, notebook;
2. **the hyphenated form**, such as daughter-in-law, master-at-arms, over-the-counter, six-pack, six-year-old, mass-produced;
3. and **the open form**, such as post office, real estate, middle class, full moon, half sister, attorney general.

13

I am not only a published writer, but I am also a trained court reporter and freelance proofreader/editor. Professionally, at least within the career of court reporting, reasons for hyphenating or not hyphenating certain words can vary. What follows are two reasons off the top of my head why one might see certain words (and even the same word) hyphenated one place and not another:

Reinforcement Learning Approaches

RLHF and other Preference Optimisation Approaches

Modeling preferences: learn a reward model

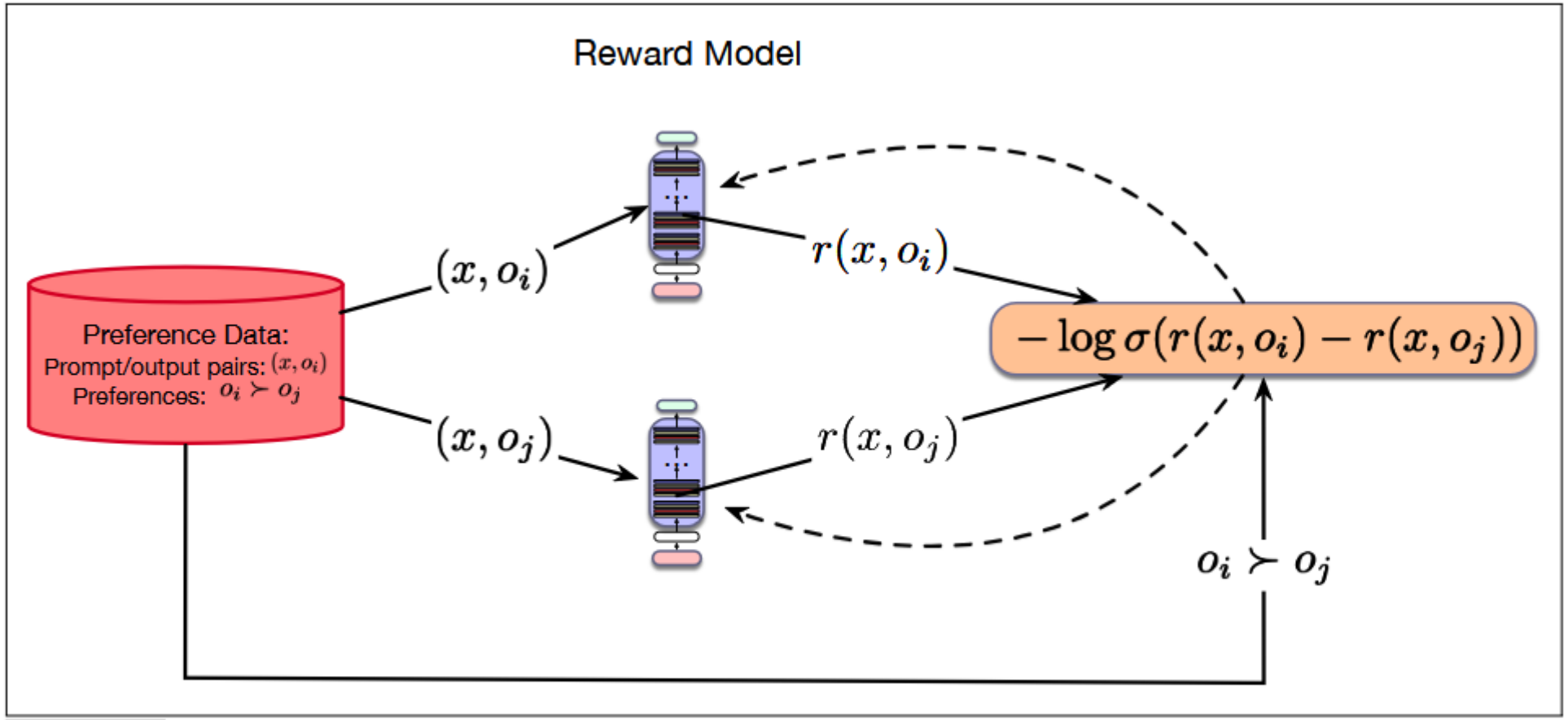


Figure 9.7 Reward model learning with a pretrained LLM. Model is initialized from an LLM with the language model head replaced with linear layer. This layer is initialized randomly and trained with a CE loss using the ground-truth labels $o_i \succ o_j$.

PPO (Proximal Policy Optimization) uses this loss to update LLM

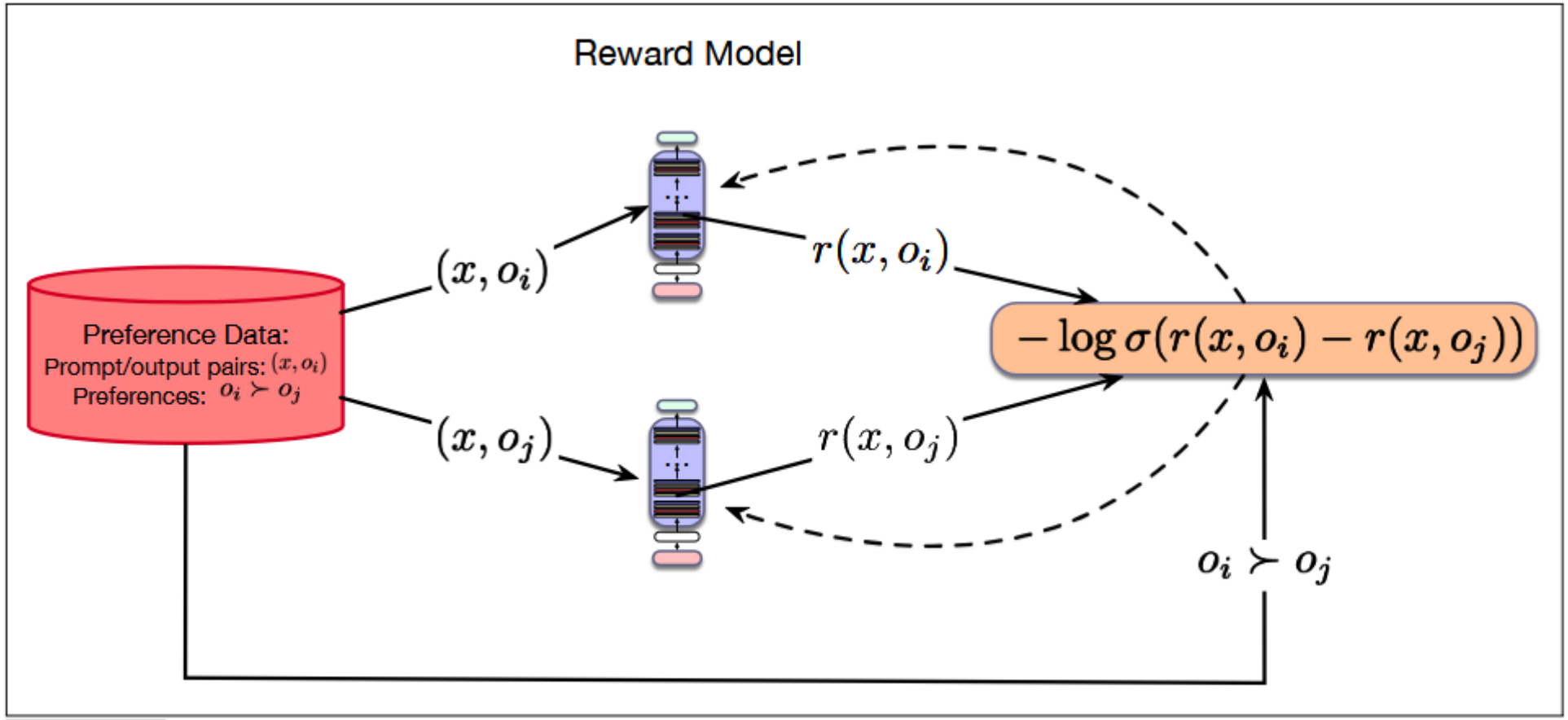


Figure 9.7 Reward model learning with a pretrained LLM. Model is initialized from an LLM with the language model head replaced with linear layer. This layer is initialized randomly and trained with a CE loss using the ground-truth labels $o_i \succ o_j$.

Direct Preference Optimization

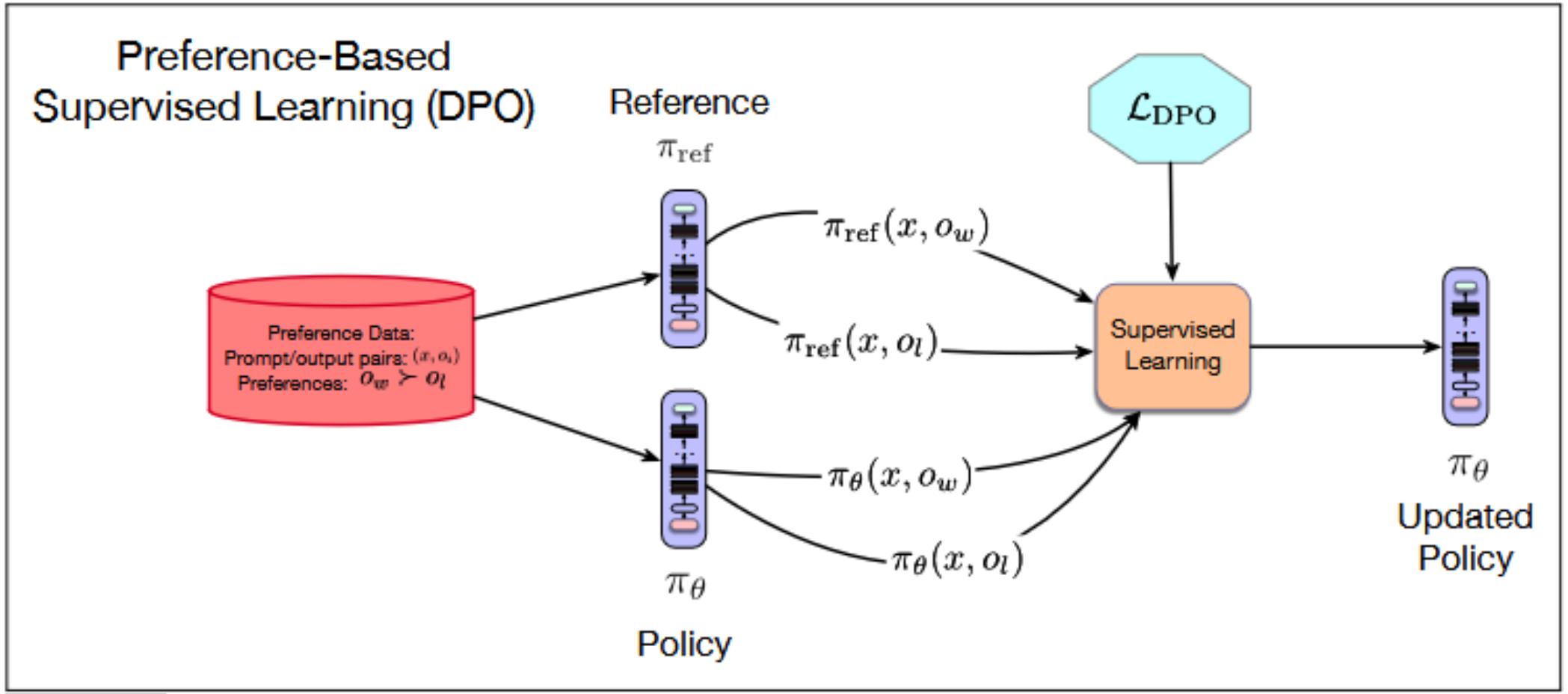


Figure 9.9 Preference-based alignment with Direct Preference Optimization.

Reinforcement Learning With Verifiable Rewards (RLVR)

- Remember AlphaGo vs AlphaZero?
- RLHF is like AlphaGo: it is RL, but the ground truth is human preferences (in that way, it is like ordinary supervised learning)
- We can also get feedback from automatically verifiable tasks: is this board state checkmate? Does this python program compile? Did this action get me points in the game?

DeepSeek: R1-Zero

"A conversation between User and Assistant. The User asks a question and the Assistant solves it. The Assistant first thinks about the reasoning process in the mind and then provides the User with the answer. The reasoning process and answer are enclosed within `<think>...</think>` and `<answer>...</answer>` tags, respectively, that is, `<think>` reasoning process here `</think><answer>` answer here `</answer>`. User: `prompt`. Assistant:", in which the `prompt` is replaced with the specific reasoning question during training. We intentionally limit our constraints to this structural format, avoiding any content-specific biases to ensure that we can accurately observe the natural progression of the model during the RL process.

DeepSeek-R1-Zero

Question: If $a > 1$, then the sum of the real solutions of $\sqrt{a-\sqrt{a+x}}=x$ is equal to

Response: <think>

To solve the equation $\sqrt{a-\sqrt{a+x}}=x$, let's start by squaring both ...

$$(\sqrt{a-\sqrt{a+x}})^2 = x^2 \Rightarrow a-\sqrt{a+x} = x^2.$$

Rearrange to isolate the inner square root term:

$$(a-x^2)^2 = a+x \Rightarrow a^2 - 2ax^2 + (x^2)^2 = a+x \Rightarrow x^4 - 2ax^2 - x + (a^2-a) = 0$$

...

Wait, wait. Wait. That's an aha moment I can flag here.

Let's reevaluate this step by step to identify whether the correct sum can be ...

We started with the equation:

$$\sqrt{a-\sqrt{a+x}} = x$$

First, let's square both sides:

$$a-\sqrt{a+x} = x^2 \Rightarrow \sqrt{a+x} = a-x^2$$

Next, I could square both sides again, treating the equation: ...

...

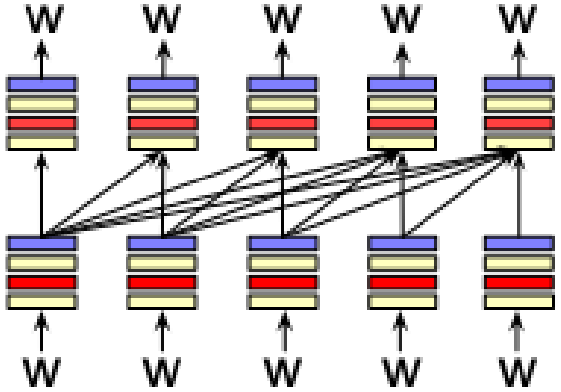
The model learns to rethink using an anthropomorphic tone. This is also an aha moment for us, allowing us to witness the power and beauty of RL.

https://www.nature.com/articles/s41586-025-09422-z/tables/1

Masked Language Models

BERT

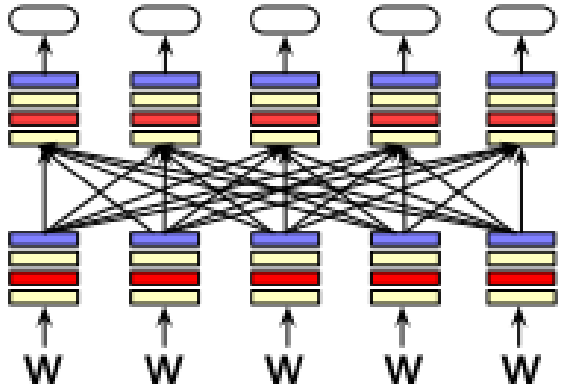
Three architectures for large language models



Decoders

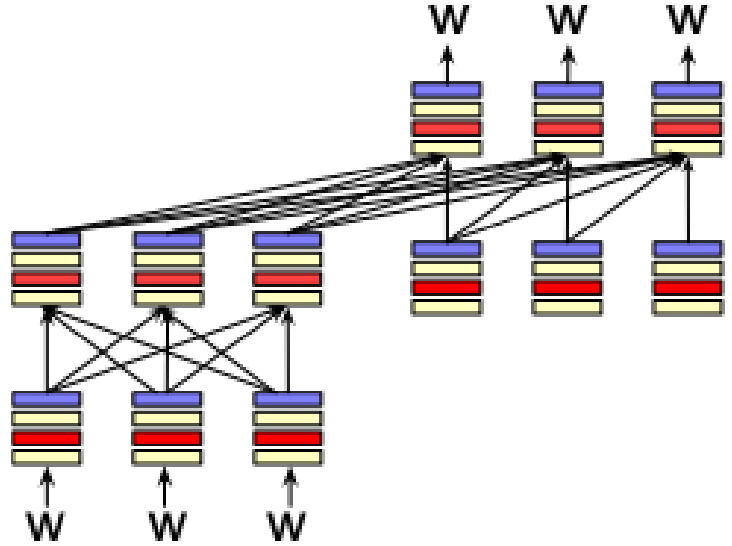
GPT, Claude, Llama

Mixtral



Encoders

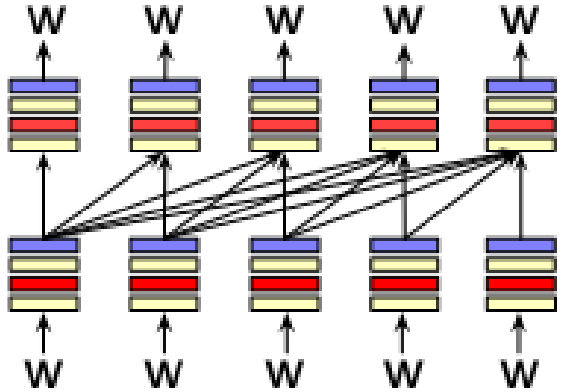
BERT family, HuBERT



Encoder-decoders

Flan-T5, Whisper

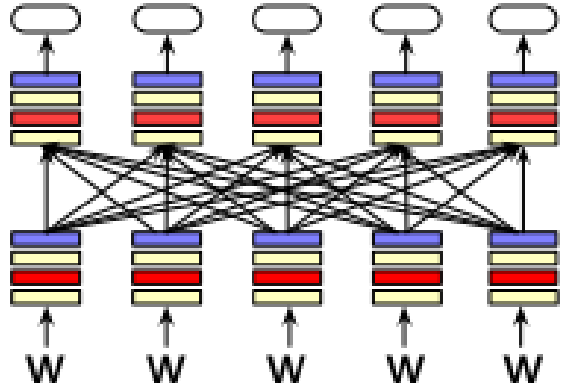
Decoders



What most people think of when we say LLM

- GPT, Claude, Llama, DeepSeek, Mistral
- A generative model
- It takes as input a series of tokens, and iteratively generates an output token one at a time.
- Left to right (causal, autoregressive)

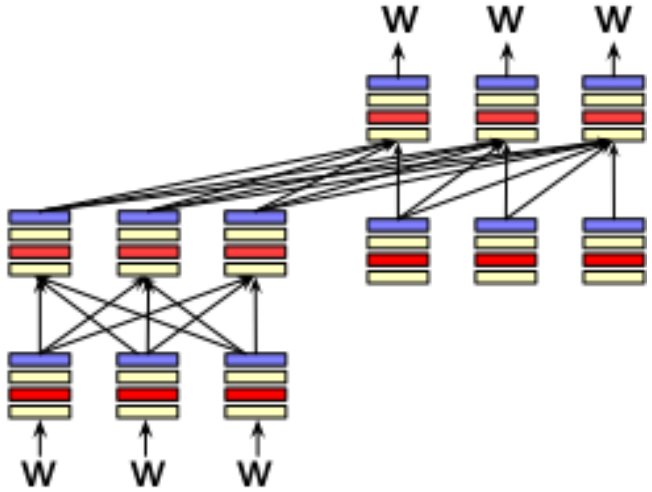
Encoders



- Masked Language Models (MLMs)
- BERT family

- Trained by predicting words from surrounding words on both sides
- Are usually **finetuned** (trained on supervised data) for classification tasks.

Encoder-Decoders

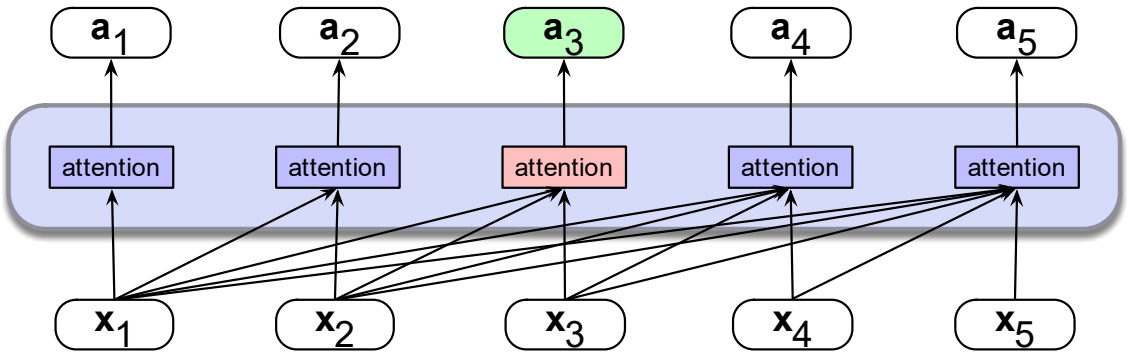


- Trained to map from one sequence to another
- Very popular for:
  - machine translation (map from one language to another)
  - speech recognition (map from acoustics to words)

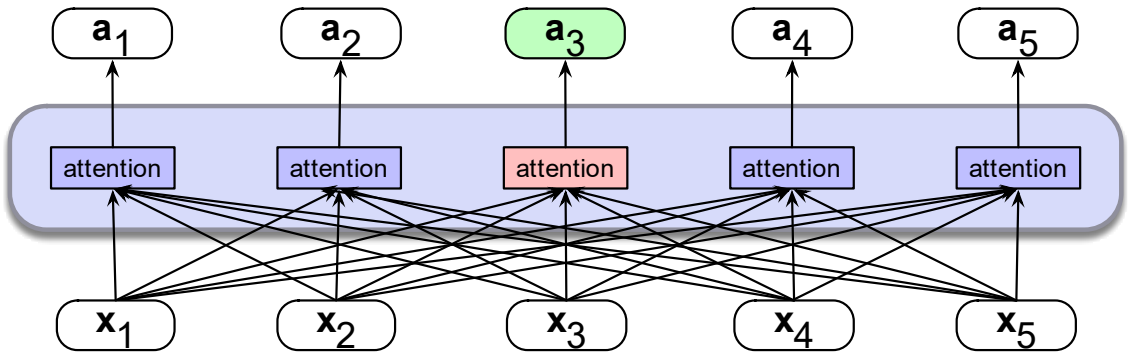
Masked Language Modeling

- We've seen autoregressive (causal, left-to-right) LMs.
- But what about tasks for which we want to peak at future tokens?
  - Especially true for tasks where we map each input token to an output token
- **Bidirectional** encoders use **masked self-attention** to
  - map sequences of input embeddings (x1,...,x*n*)
  - to sequences of output embeddings of the same length (h1,...,h*n*),
  - where the output vectors have been contextualized using information from the entire input sequence.

Bidirectional Self-Attention



a) A causal self-attention layer



b) A bidirectional self-attention layer

Easy! We just remove the mask

Casual self-attention

N

q1·k1	−∞	−∞	−∞
q2·k1	q2·k2	−∞	−∞
q3·k1	q3·k2	q3·k3	−∞
q4·k1	q4·k2	q4·k3	q4·k4

N

$$\mathbf{head} = \text{softmax}\left(\text{mask}\left(\frac{\mathbf{QK}^\intercal}{\sqrt{d_k}}\right)\right)\mathbf{V}$$

Bidirectional self-attention

N

q1·k1	q1·k2	q1·k3	q1·k4
q2·k1	q2·k2	q2·k3	q2·k4
q3·k1	q3·k2	q3·k3	q3·k4
q4·k1	q4·k2	q4·k3	q4·k4

N

$$\mathbf{head} = \text{softmax}\left(\frac{\mathbf{QK}^\intercal}{\sqrt{d_k}}\right)\mathbf{V}$$

BERT: Bidirectional Encoder Representations from Transformers

BERT (Devlin et al., 2019)

- 30,000 English-only tokens (WordPiece tokenizer)
- Input context window *N*=512 tokens, and model dimensionality *d*=768
- *L*=12 layers of transformer blocks, each with *A*=12 (bidirectional) multihead-attention layers.
- The resulting model has about 100M parameters.

XLM-RoBERTa (Conneau et al., 2020)

- 250,000 multilingual tokens (SentencePiece Unigram LM tokenizer)
- Input context window *N*=512 tokens,model dimensionality *d*=1024
- *L*=24 layers of transformer blocks, with *A*=16 multihead attention layers each
- The resulting model has about 550M parameters.

Masked Language Models

BERT

Masked Language Models

Masked LM training

Masked training intuition

- For **left-to-right LMs**, the model tries to predict the last word from prior words:

  The water of Walden Pond is so beautifully ____

  - And we train it to improve its predictions.
- For **bidirectional masked LMs**, the model tries to predict one or more words from all the rest of the words:

  The ____ of Walden Pond ____ so beautifully blue

  - The model generates a probability distribution over the vocabulary for each missing token
  - We use the cross-entropy loss from each of the model's predictions to drive the learning process.

MLM training in BERT

15% of the tokens are randomly chosen to be part of the masking

Example: "Lunch was **delicious**", if delicious was randomly chosen:

Three possibilities:

1. 80%: Token is replaced with special token [MASK]

   Lunch was **delicious ->** Lunch was **[MASK]**

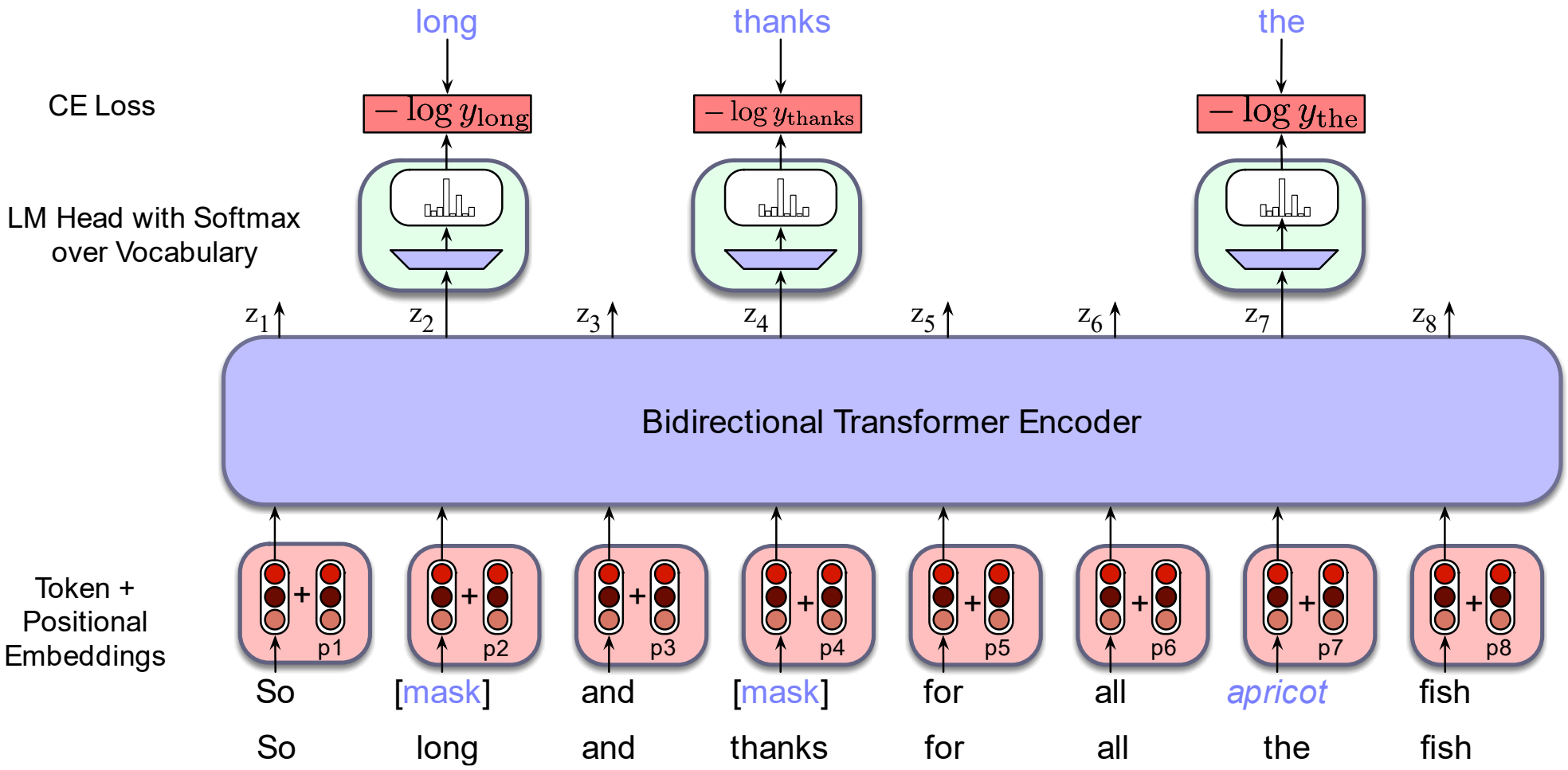
2. 10%: Token is replaced with a random token (sampled from unigram prob)

   Lunch was **delicious ->** Lunch was **gasp**

3. 10%: Token is unchanged

   Lunch was **delicious ->** Lunch was **delicious**

In detail


long
thanks
the
CE Loss
$-\log y_{\text{long}}$
$-\log y_{\text{thanks}}$
$-\log y_{\text{the}}$
LM Head with Softmax over Vocabulary
z_1 z_2 z_3 z_4 z_5 z_6 z_7 z_8
Bidirectional Transformer Encoder
Token + Positional Embeddings
p1 p2 p3 p4 p5 p6 p7 p8
So [mask] and [mask] for all apricot fish
So long and thanks for all the fish

Interactive tokenizer example:

https://platform.openai.com/tokenizer

MLM loss

The LM head takes output of final transformer layer L, multiplies it by unembedding layer and turns into probabilities:

$$\mathbf{u}_i = \mathbf{h}_\mathbf{i}^\mathbf{L}\ \mathbf{E}^\mathbf{T}$$

$$\mathbf{y}_i = \text{softmax}(\mathbf{u_i})$$

E.g., for the x_i corresponding to "long", the loss is the probability of the correct word *long*, given output h^L_i):

$$L_{MLM}(x_i) = -\log P(x_i|\mathbf{h}_i^L)$$

We get the gradients by taking the average of this loss over the batch

$$L_{MLM} = -\frac{1}{|M|}\sum_{i\in M}\log P(x_i|\mathbf{h}_i^L)$$

Next Sentence Prediction

Given 2 sentences the model predicts if they are a real pair of adjacent sentences from the training corpus or a pair of unrelated sentences.

BERT introduces two special tokens

- [CLS] is prepended to the input sentence pair,
- [SEP] is placed between the sentences, and also after second sentence

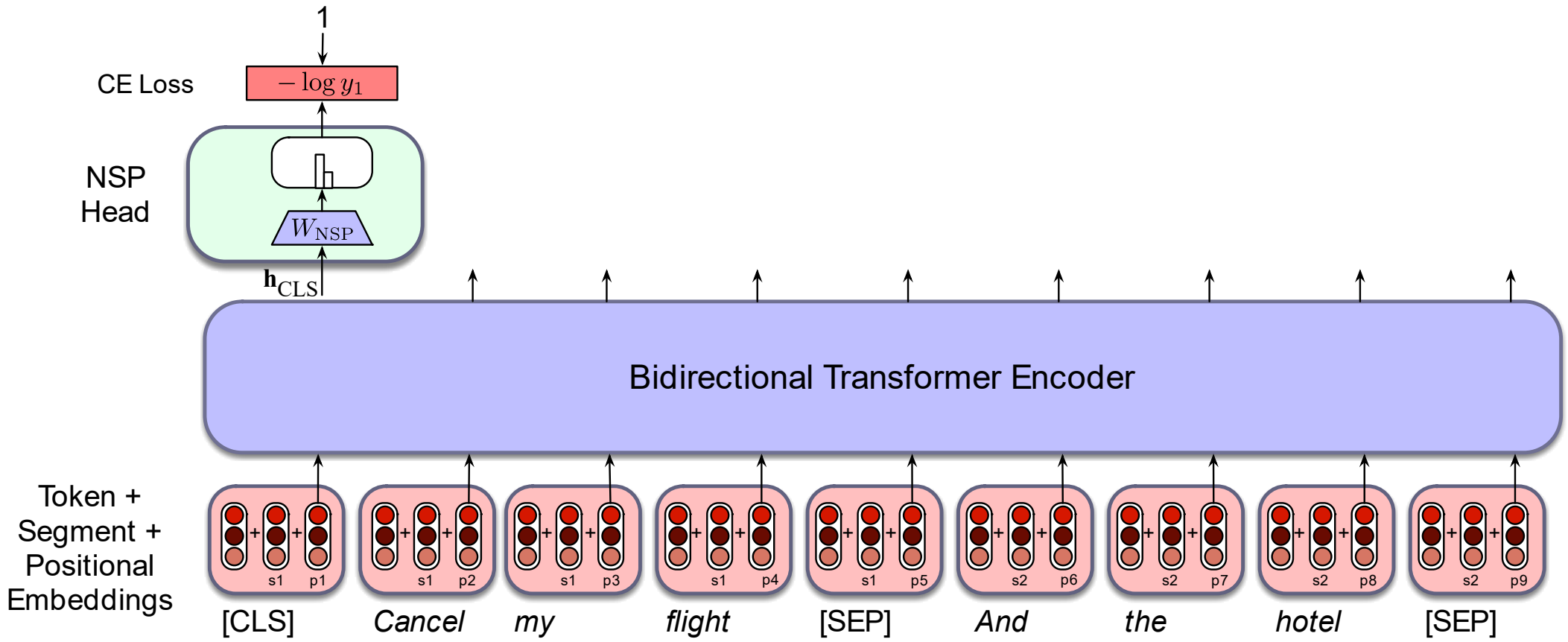
And two more special tokens

- [1st segment] and [2nd segment]
- These are added to the input embedding and positional embedding

h^L_{CLS} from the final layer [CLS] token is input to classifier head (weights W_{NSP}) that predicts two classes:.

$$y_i = \text{softmax}(h^L_{CLS} W_{NSP})$$

NSP Loss with classification head



More details

Original model was trained with 40 passes over training data

Some models (like RoBERTa) drop NSP loss

Tokenizer for multilingual models is trained from stratified sample of languages (some data from each language)

Multilingual models are better than monolingual models with small numbers of languages

- With large numbers of languages, monolingual models in that language can be better
- The "curse of multilinguality"

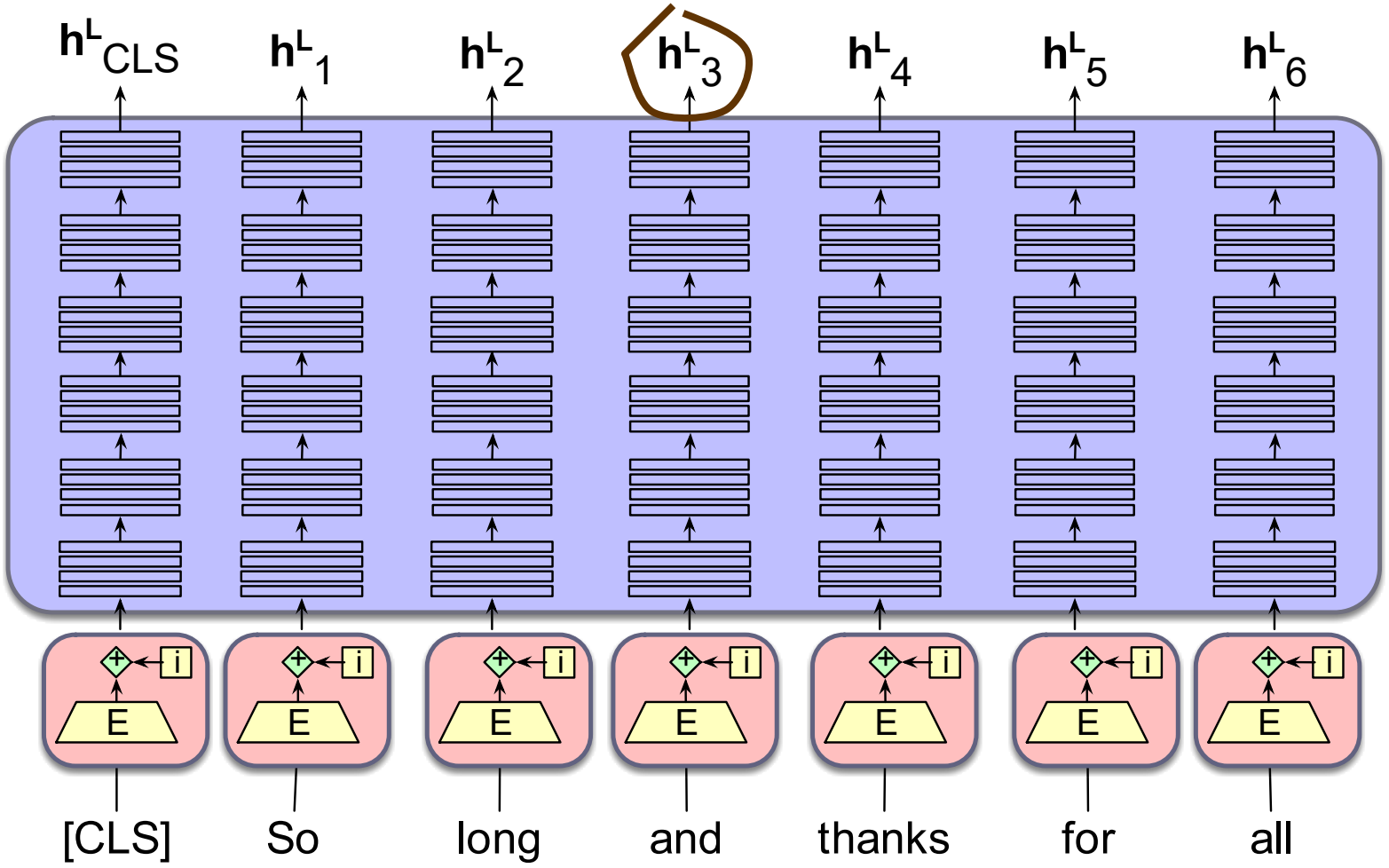
Masked Language Models

Masked LM training

Masked Language Models

Contextual Embeddings

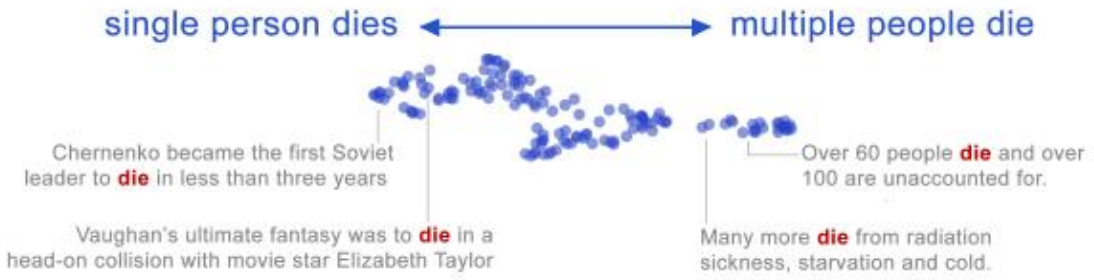
Contextual Embeddings to represent words



Static vs Contextual Embeddings

Static embeddings represent **word types** (dictionary entries)

Contextual embeddings represent **word instances** (one for each time the word occurs in any context/sentence)



Word sense

Words are ambiguous

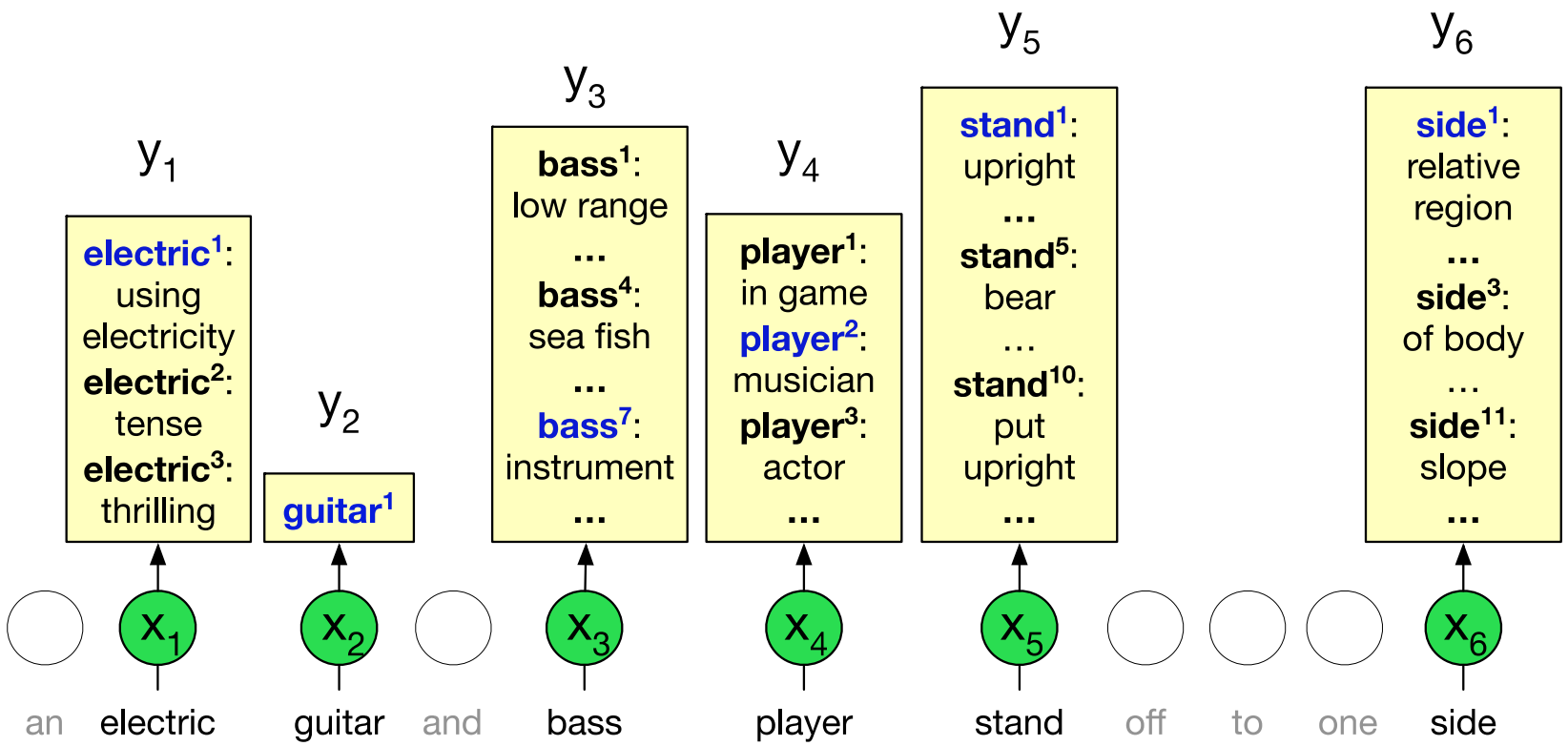
A **word sense** is a discrete representation of one aspect of meaning

mouse^{1} : .... a *mouse* controlling a computer system in 1968.
mouse^{2} : .... a quiet animal like a *mouse*
bank^{1} : ...a *bank* can hold the investments in a custodial account ...
bank^{2} : ...as agriculture burgeons on the east *bank*, the river ...

Contextual embeddings offer a continuous high-dimensional model of meaning that is more fine grained than discrete senses.

Word sense disambiguation (WSD)

The task of selecting the correct sense for a word.



1-nearest neighbor algorithm for WSD

Melamud et al (2016), Peters et al (2018)

At training time, take a sense-labeled corpus like SEMCOR

Run corpus through BERT to get contextual embedding for each token

- E.g., pooling representations from last 4 BERT transformer layer

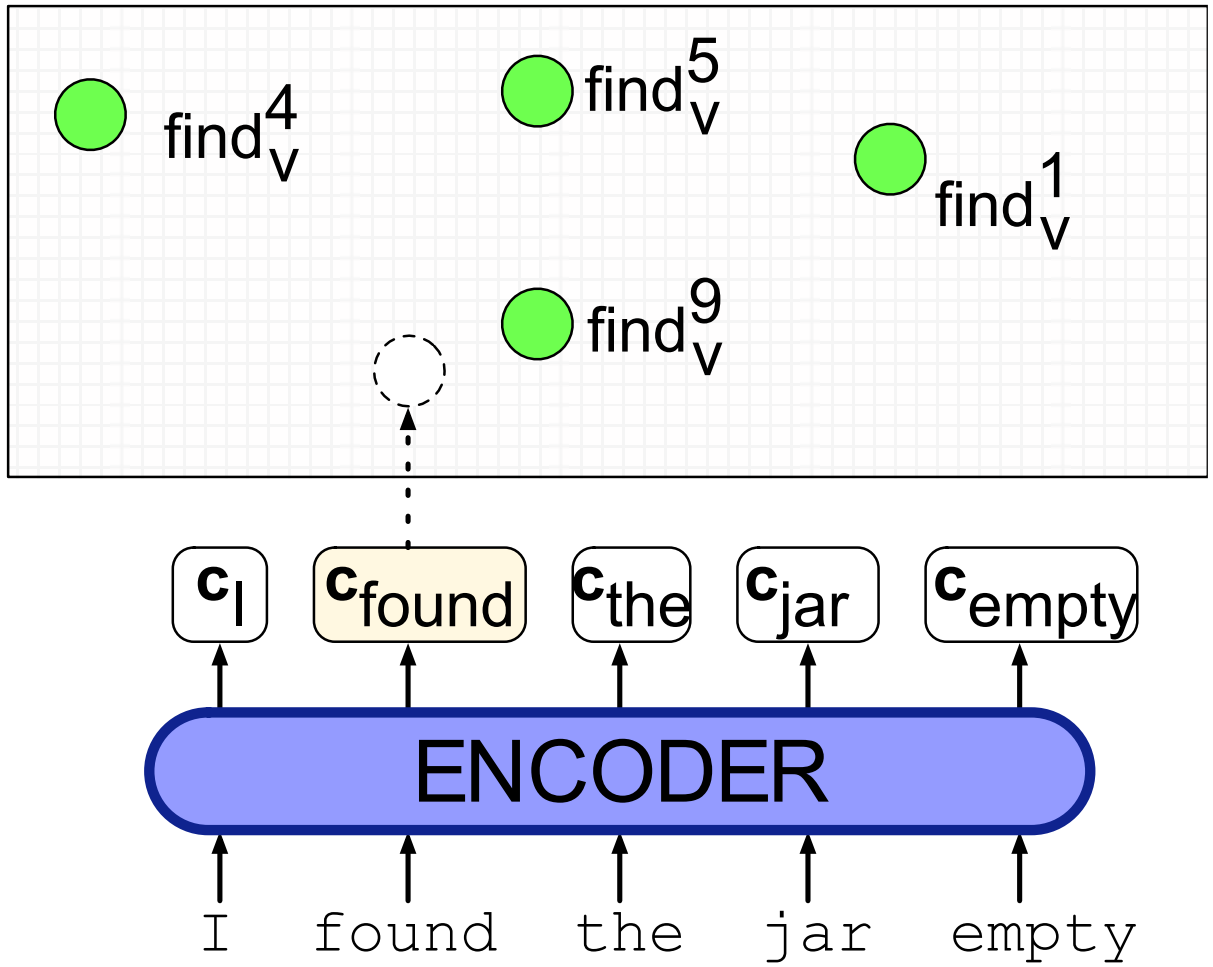
Then for each sense s of word w for n tokens of that sense, pool embeddings:

$$\mathbf{v}_s = \frac{1}{n}\sum_i \mathbf{v}_i \qquad \forall \mathbf{v}_i \in \text{tokens}(s)$$

At test time, given a token of a target word *t*, compute contextual embedding t and choose its nearest neighbor sense from training set

$$\text{sense}(t) = \underset{s \in \text{senses}(t)}{\text{argmax}}\ \text{cosine}(\mathbf{t}, \mathbf{v}_s)$$

1-nearest neighbor algorithm for WSD



Similarity and contextual embeddings

- We generally use cosine as for static embeddings
- But some issues:
  - Contextual embeddings tend to be **anisotropic:** all point in roughly the same direction so have high inherent cosines (Ethayarajh 2019)
  - Cosine measure are dominated by a small number of "rogue" dimensions with very high values (Timkey and van Schijndel 2021)
  - Cosine tends to underestimate human judgments on similarity of word meaning for very frequent words (Zhou et al., 2022)

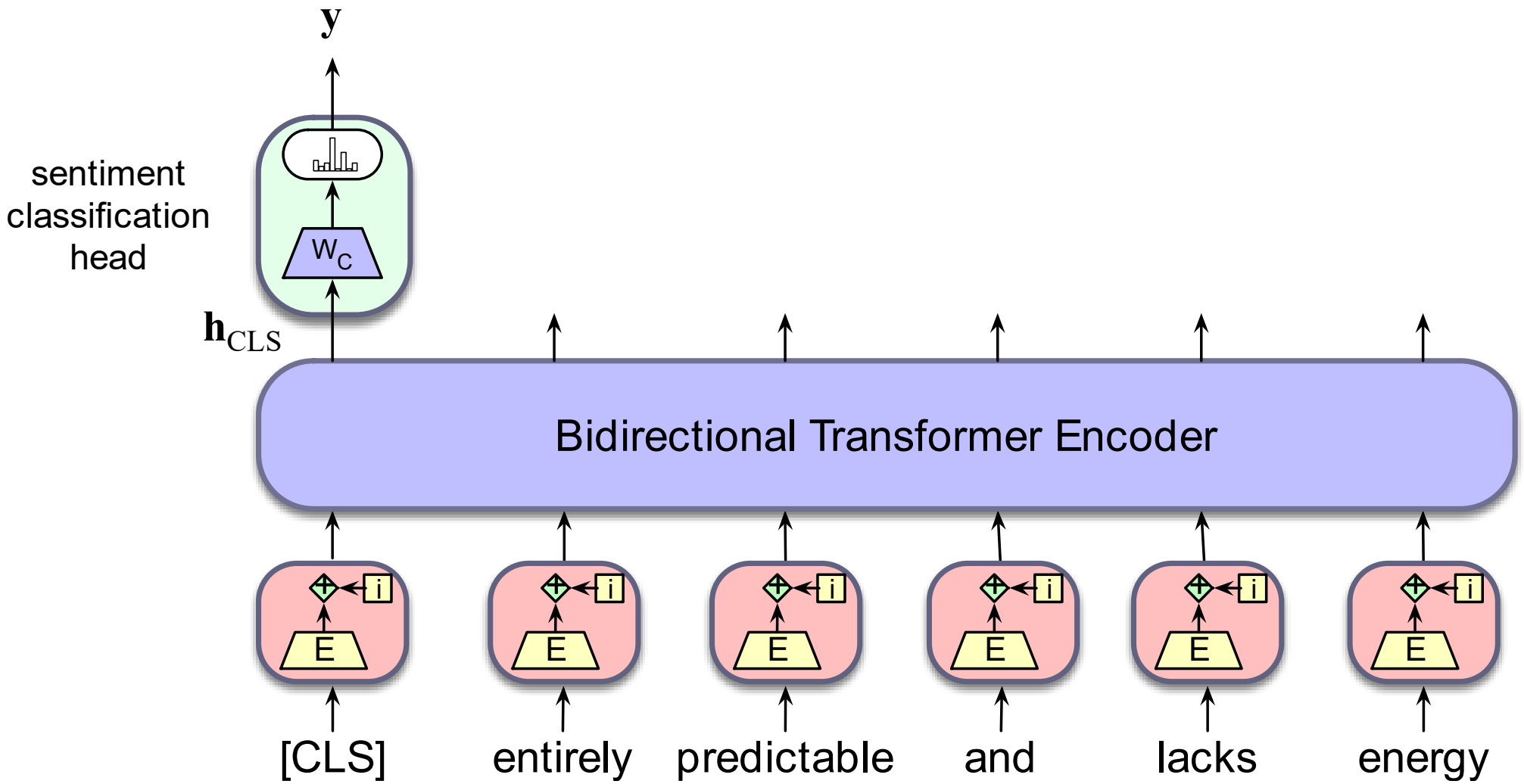
Masked Language Models

Contextual Embeddings

Masked Language Models

Fine-Tuning for Classification

Adding a sentiment classification head



Sequence-Pair classification

Assign a label to pairs of sentences:

- paraphrase detection (are the two sentences paraphrases of each other?)
- logical entailment (does sentence A logically entail sentence B?)
- discourse coherence (how coherent is sentence B as a follow-on to sentence A?)

Example: Natural Language Inference

Pairs of sentences are given one of 3 labels

- `Neutral`
  - a: Jon walked back to the town to the smithy.
  - b: Jon traveled back to his hometown.
- `Contradicts`
  - a: Tourist Information offices can be very helpful.
  - b: Tourist Information offices are never of any help.
- `Entails`
  - a: I'm confused.
  - b: Not all of it is very clear to me.

Algorithm: pass the premise/hypothesis pairs through a bidirectional encoder and use the output vector for the [CLS] token as the input to the classification head .

Fine-tuning for sequence labeling

Assign a label from a small fixed set of labels to each token in the sequence.

- Named entity recognition
- Part of speech tagging

.

Named Entity Recognition

A **named entity** is anything that can be referred to with a proper name: a person, a location, an organization

Named entity recognition (NER): find spans of text that constitute proper names and tag the type of the entity

Type	Tag	Sample Categories	Example sentences
People	PER	people, characters	**Turing** is a giant of computer science.
Organization	ORG	companies, sports teams	The **IPCC** warned about the cyclone.
Location	LOC	regions, mountains, seas	**Mt. Sanitas** is in **Sunshine Canyon**.
Geo-Political Entity	GPE	countries, states	**Palo Alto** is raising the fees for parking.

Named Entity Recognition

Citing high fuel prices, [ORG **United Airlines**] said [TIME **Friday**] it has increased fares by [MONEY **$6**] per round trip on flights to some cities also served by lower-cost carriers. [ORG **American Airlines**], a unit of [ORG **AMR Corp.**], immediately matched the move, spokesman [PER **Tim Wagner**] said. [ORG **United**], a unit of [ORG **UAL Corp.**], said the increase took effect [TIME **Thursday**] and applies to most routes where it competes against discount carriers, such as [LOC **Chicago**] to [LOC **Dallas**] and [LOC **Denver**] to [LOC **San Francisco**].

BIO Tagging

Ramshaw and Marcus (1995)

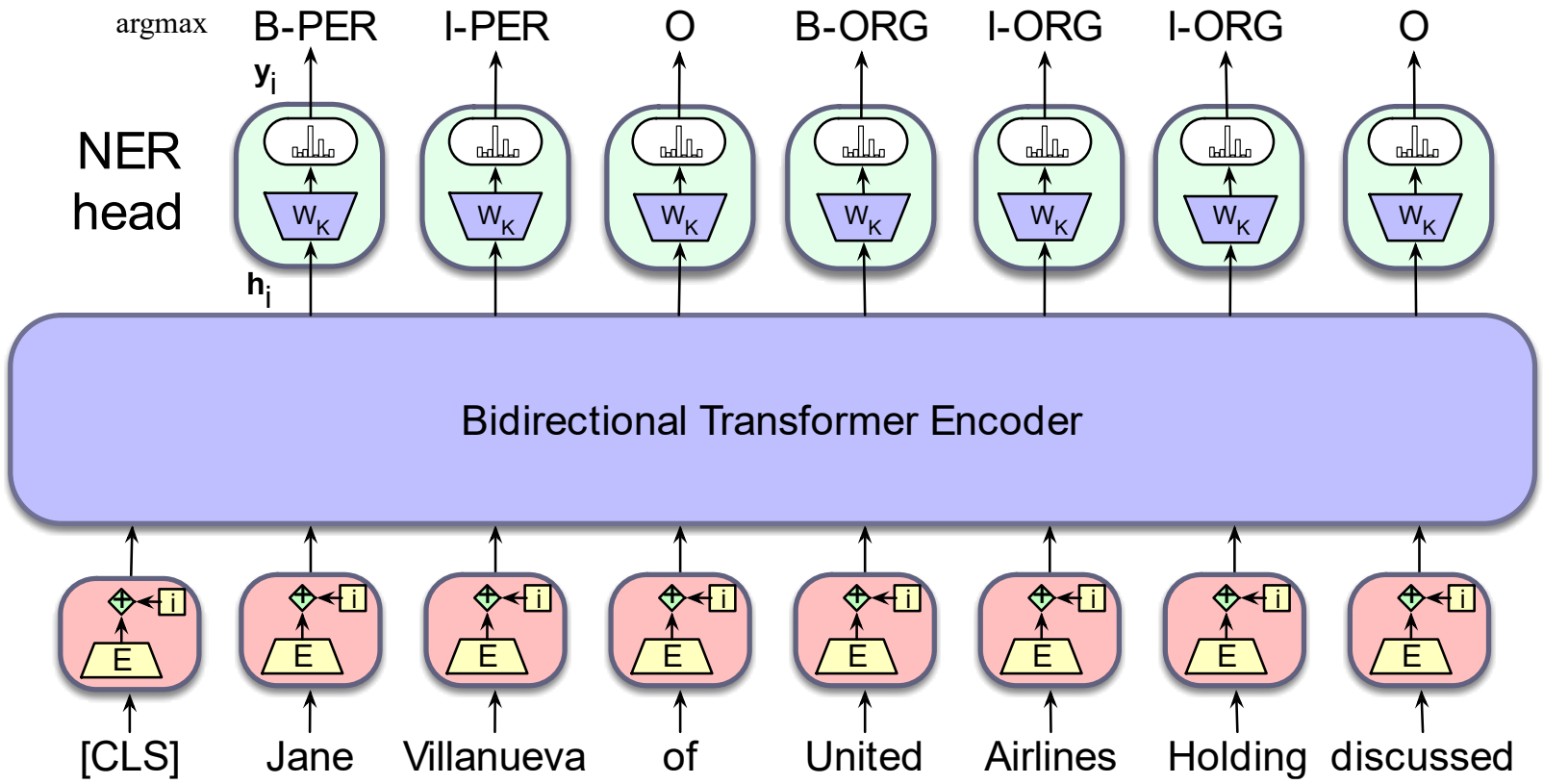
A method that lets us turn a segmentation task (finding boundaries of entities) into a classification task

[PER **Jane Villanueva**] of [ORG **United**] , a unit of [ORG **United Airlines Holding**] , said the fare applies to the [LOC **Chicago**] route.

Words	IO Label	BIO Label	BIOES Label
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

Sequence labeling

$$\mathbf{y_i} = \text{softmax}(\mathbf{h_i^L W_K})$$
$$\mathbf{t_i} = \text{argmax}_k(\mathbf{y}_i)$$



More details

We need to map between tokens (used by LLM) and words (used in definition of name entities)

We evaluate NER with F1 (precision/recall)

Masked Language Models

Fine-Tuning for Classification